

# Setting up a data science project

For small personal projects, notebooks and scripts can be stored flatly in a simple directory. However, as your project grows in complexity, it requires a dedicated directory that stores input and output data, ETL and training scripts, analysis notebooks, and configuration files.

Furthermore, this directory can also store the Python packages the project depends on (a *virtual environment*), version control indexes, dockerfile, and other development files.

## Setting up the project directory

An example of a minimal data science project directory:

```
my_project/
├── data/
│   ├── raw/
│   └── processed/
├── notebooks/
├── src/
├── outputs/
├── .gitignore
├── requirements.txt
└── README.md
```

**data/** holds the data files used in your project.

- **data/raw/** : files of raw data. They should be kept exactly as they are and not modified by the data transformation steps.
- **data/processed/** : results of data processing. Missing values imputed, duplicates removed, new columns added, data made ready for training.

**outputs/** can be used to store the artifacts generated by the code, such as trained models or reports.

**notebooks/** stores the notebooks created for data exploration, experiments, and reports.

Use long and descriptive names for notebooks.

Instead of

Untitled123.ipynb

use

2026-01-04-mko-Visualize Customer Data.ipynb

By starting the name with the date in YYYY-MM-DD format, we can easily sort the files by date. Initials help identify the creator if multiple users have access to the directory.

`src/` : Python code files for your project. For example, it may contain the following:

```
src/
├── __init__.py
├── config.py
├── predict.py
├── process.py
├── train.py
└── utils.py
```

The `__init__.py` file sets up this directory as a Python package, which is useful for installing your project like any other Python package and import variables from it:

```
from my_project.utils import fancyfunction
```

More on this later.

Alternatively, some Python code can be moved to another directory, say, `scripts/`. Then `src/` stores only the most general code library for I/O and processing, and `scripts/` holds more special-purpose code, such as report generation.

`config.py` can be used to set up general configuration options for your code, such as data paths.

```
# config.py
from pathlib import Path

SRC_DIR = Path(__file__).resolve().parent
PROJECT_ROOT = SRC_DIR.parent
RAW_DATA_DIR = PROJECT_ROOT / "data" / "raw"
PROCESSED_DATA_DIR = PROJECT_ROOT / "data" / "processed"

CUSTOMER_DB_URL = 'sql:///customer/db/url'
PURCHASES_DB_URL = 'sql:///purchases/db/url'
```

Then, we can use this in other scripts or notebooks

```
# notebook.ipynb
from my_project.config import RAW_DATA_DIR
import pandas as pd

df = pd.read_csv(RAW_DATA_DIR / "test.csv")
```

`requirements.txt` is the conventional name for a file that lists the Python packages your project uses. It is used for installing all packages to the virtual environment.

`.gitignore` lists files and directories that should not be tracked by *git*. For example, `data/` is not tracked; its contents should be automatically reproducible.

`README.md` is a free-form Markdown file explaining the project.

Other possible files to include:

- *Makefile*: Used by the `make` utility in order to generate some files automatically. For example, the commands to download raw data files and generate processed data files can be placed here.
- *Dockerfile*: Describes the Docker image for containerizing this application.
- *pyproject.toml*: The current standard for setting up Python packages.
- *.env*: Stores some shell variable assignments. Tools like `dotenv` can read them programmatically. Good for storing secret keys and passwords. Must be listed in `.gitignore`.

## Virtual environments and package management

Different projects may rely on different libraries that are incompatible with each other. Even if they use the same library, newer versions may not be compatible with older versions. If you install all of them globally, eventually some of your projects will cease to run.

A *virtual environment* is an isolated Python installation, local to a specific project. It has its own copy of the Python interpreter and its own `site-packages` directory where libraries are installed. Critically, it is completely independent of your system-wide Python and of every other virtual environment on your machine.

### Creating an environment

Python has the built-in `venv` module which can be used to create a virtual environment in a project directory.

In the project directory, run the following command in the terminal:

```
python -m venv .venv
```

This creates a directory called `.venv` containing a self-contained Python installation. The name `.venv` is a widely adopted convention; the leading dot keeps it hidden in most file browsers. You can name it anything you like, but be consistent.

### Activating an environment

Before you can use the environment, you must *activate* it. Activation modifies your shell's `PATH` so that `python` and `pip` commands resolve to the versions inside `.venv` rather than your system-wide Python.

On Linux and MacOS:

```
source .venv/bin/activate
```

On Windows:

```
.venv\Scripts\activate
```

After activation, your shell prompt will be prefixed with `(.venv)` as a visual reminder that the environment is active. You can verify which Python is in use with `which python` (or `where python` in Windows).

## Installing packages

`pip` is the package manager that comes with Python. It is used for installing third-party packages from the PyPI (Python Package Index)

With the environment active, pip installs packages exclusively into `.venv`. Your system Python and all other projects remain untouched.

```
pip install pandas scikit-learn matplotlib jupyterlab
```

## Deactivating the environment

When you are done working on a project, deactivate the environment to return to your system Python:

```
deactivate
```

## Deleting the environment

To remove the environment, simply delete the directory `.venv`. Repeat the steps to create it again.

## Pinning dependencies with requirements.txt

Installing packages is only half the story. The other half is recording exactly what you installed, so that the environment can be recreated exactly.

After installing your dependencies, generate a requirements file. It is conventionally named `requirements.txt`.

```
pip freeze > requirements.txt
```

`pip freeze` outputs every installed package and its exact version. The result looks something like this:

```
matplotlib==3.9.0  
numpy==2.0.1
```

```
pandas==2.2.2
scikit-learn==1.5.1
```

To recreate this environment from scratch on another machine:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

There can be more than requirements file in a project. For example, `requirements-dev.txt` can be used to store packages that are only needed for development. If you need Jupyter and plotly for interactive data exploration, but the final project does not use them, the file can contain:

```
ipykernel
jupyter
plotly
```

Then, you run

```
pip install -r requirements.txt
pip install -r requirements-dev.txt
```

When we create the requirements file manually, we can omit version numbers. Then, the environment uses the latest versions of the packages. However, when strict reproducibility is required, this is not recommended.

One can also specify a *minimum* version of each package. The line

```
matplotlib>=3.9
```

means "use version 3.9 or higher".

## Setting up your project as a package with pyproject.toml

The configuration file `pyproject.toml` is the current standard for setting up Python packages. It collects metadata about the dependencies, dev tools, linters, formatters, and test configurations. It removes the clutter resulting from having separate configuration files for each tool.

This file, like the requirements file, is located at the project root.

Reference: <https://packaging.python.org/en/latest/guides/writing-pyproject-toml/#>

An example `pyproject.toml` file:

```
[build-system]
requires = ["setuptools>=82.0.0"]
build-backend = "setuptools.build_meta"
```

```

[project]
name = "my-ds-project"
version = "0.1.0"
description = "Churn prediction pipeline"
requires-python = ">=3.10"

authors = [
    {name = "Pradyun Gedam", email = "pradyun@example.com"},
    {name = "Tzu-Ping Chung", email = "tzu-ping@example.com"},
]

readme = "README.md"

# Core runtime dependencies
dependencies = [
    "pandas>=2.0",
    "scikit-learn>=1.3",
    "matplotlib",
]

# Optional dependency groups
[project.optional-dependencies]
dev = [
    "pytest",
    "black",
    "ruff",
]
notebooks = [
    "jupyterlab",
    "ipywidgets",
]

```

To install required packages, we give the command:

```
pip install .
```

To install all dependencies:

```
pip install .[dev,notebooks]
```

## Editable local installation

When we put source files and notebooks into separate directories, Python's import system cannot find modules.

To work around this, we can add the `src` directory to the python search path:

```
import sys
sys.path.append("../src")
```

However, this is a fragile solution. A cleaner solution is to install your project as a local package.

Run the following command at the project root. Don't forget the period (.) at the end.

```
pip install -e .
```

`pip` now installs the project and its dependencies in the active virtual environment.

The `-e` flag makes this an *editable* install. When the source code is changed and saved, the changes are applied the next time the module is loaded. There is no need to update packages manually with `pip install --update`.

## Creating executable scripts

Suppose we want to create an executable based on a function in the module. This can be specified in `pyproject.toml` in the `[project.scripts]` section.

```
[project.scripts]
```

```
myapp = "myproj:cli"
```

This specifies that an executable with the name `myapp` is to be created, which runs the function `cli()` in the module `myproj`.

```
# src/myproj.py
```

```
def cli():  
    print("Congratulations. Your app is running!")
```

After installing again, we can run this executable:

```
$ pip install -e .
```

```
$ myapp
```

```
Congratulations. Your app is running!
```

```
This executable is located at .venv/bin
```

## Executable scripts with command-line arguments

Add another executable named `add`, which adds up numbers given at command line.

```
[project.scripts]
```

```
myapp = "myproj:cli"
```

```
add = "myproj:add"
```

Create the relevant function in the source code:

```
# src/myproj.py
```

```
import sys
```

```
def cli():  
    print("Congratulations. Your app is running!")
```

```
def add():  
    print(sum([int(_) for _ in sys.argv[1:])))
```

Reinstall and run:

```
$ pip install -e .
```

```
$ add 10 20 30
```

```
60
```

## Managing projects with `uv`

`uv` is a newer tool with several strong points:

- Combines virtual environment generation and package management in a single tool.
- Much faster than pip.
- Allows selecting the Python interpreter version, not just package versions.
- Easy to create "throwaway" environments for single use cases.

Main reference: <https://docs.astral.sh/uv/>

To install, follow the steps that is suitable for your system given at: <https://docs.astral.sh/uv/getting-started/installation/>

## Initialize a project

Once `uv` is installed, we can create a project

```
$ uv init ds-project
```

This command creates the directory `ds-project` and creates some files, including a `pyproject.toml` file. Also initializes `git`.

```
$ cd ds-project
$ ls -a
.   .git          .python-version  main.py
..  .gitignore    README.md        pyproject.toml
```

## Specify a Python version

At initialization we can choose one specific Python version.

```
$ uv init ds-project --python 3.12
```

## Add dependencies

We can add packages to the project with the `uv add` command. If a virtual environment is not present, it is created.

```
$ uv add pandas numpy matplotlib
```

After the installation, `uv` updates `pyproject.toml` automatically.

To remove packages:

```
$ uv remove matplotlib
```

## Running `uv` with existing `pyproject.toml`

You may have the project directory already set up with a `pyproject.toml` in place, just like the `my_project` in the previous section. To set up the virtual environment and to build the package, you run `uv sync`.

```
$ cd ~/my_project
$ uv sync
```

This provides an editable install as before. Entry points also run as expected.



```
$ source .venv/bin/activate
$ run add 1 2 3
6
```

## Running scripts

The `uv run` command is used for running scripts and specific entry points. It executes the relevant virtual environment of the script.

Our source code:

```
# src/myproj.py
def cli():
    print("Congratulations. Your app is running!")

import sys
def add():
    print(sum([int(_) for _ in sys.argv[1:]]))

if __name__ == "__main__":
    print("Hello world!")
```

Running on the terminal:

```
$ uv run src/myproj.py # venv activated automatically
Hello world!
$ uv run myapp # entrypoint from pyproject.toml
Congratulations. Your app is running!
$ uv run add 1 2 3 # entrypoint from pyproject.toml
6
```

## Exercise

- Create a data science project named "wholesale\_analysis". Use `uv` to initialize the project directory. Manually add other relevant subdirectories.
- Add `pandas` dependency.
- Download the data file *Wholesale Customers Data* (<https://gist.github.com/mkozturk/edc44c266efb999517fd38390b8f8dc1>) and store it in `data/raw/`
- Create a file `src/analyze.py`. In it, create a function `total_revenue(region, channel)` that returns the total revenue from a given region and channel, broken by item types.
- Create a file `src/cli.py`. In it, create a function `display_report()` which calls `total_revenue()` and prints the result. The region and channel input should be read as command-line arguments. Add this function as an entrypoint to `pyproject.toml` with the name `revenue_report`.
- Update the project with `uv sync`.
- Run the command with `uv run revenue_report 1 1`.